

TigerGraph-Powered Agentic Fraud Investigation System

End-to-End Technical & Engineering Progress Report — HHGOA Hackathon (Track 5)

AUTHORITATIVE BACKEND: TIGERGRAPH LIVE

SAVANNA CLOUD v4.2.5 (AWS US-EAST-1)

DATASET: IEEE-CIS (590K TXNS)

PHASE 1: 100% OK / ALL PASSED

8 Vertices

GSQL GRAPH SCHEMA

12 Edges

DIRECTED + REVERSE

6 Queries

COMPILED C++ GSQL

10,000+

LIVE INGESTED ENTITIES

100% Pass

8/8 UNIT TESTS

1. Executive Summary & Architectural Vision

The objective of this project is to build an **analyst-grade Financial Crime Investigation Workstation** for the **HHGOA Hackathon**, driven by an autonomous multi-step **Agentic Investigation Loop** and backed by an authoritative **TigerGraph graph database**.

Traditional fraud systems rely on siloed rule engines or isolated black-box tabular classifiers that flag suspicious events without investigating the relational context. This project implements an intelligent, graph-native workflow that mirrors a senior Financial Intelligence Unit (FIU) analyst:

- Case Initiation & Trigger Evaluation:** Ingesting transaction alerts (e.g. out-of-region card present, micro-authorizations, new device login).
- Graph Relational Traversal:** Traversing multi-hop connections across cards, customers, devices, proxies, email domains, and billing regions in live TigerGraph.
- Deterministic Policy Compliance (R1–R10):** Applying explicit banking risk rules with tiered escalation (Automated resolution, Level-1 AML approval, Level-2 Compliance sign-off).
- GraphRAG Memory Retrieval:** Matching target typologies against 5,565 historical closed cases stored directly in TigerGraph.
- SAR Generation & Writeback:** Generating FinCEN-compliant Suspicious Activity Reports (SARs) and recording the dynamic `InvestigationCase` back into TigerGraph graph memory.

Authoritative Architectural Rule

TigerGraph is the sole authoritative backend. The local SQLite/NetworkX engine is strictly an offline development fallback and never masquerades as TigerGraph. A persistent health status badge (`TIGERGRAPH_LIVE` vs `LOCAL_FALLBACK`) is maintained at all times.

2. Authoritative Datasets & Data Engineering

The system is constructed using the authoritative IEEE-CIS (Vesta) fraud benchmark dataset:

Dataset Component	Volume & Dimensions	Key Fields & Data Signatures	Role in Architecture
transactions.csv	590,742 records (707 MB)	<code>TransactionID</code> , <code>customer_id</code> , <code>card1..6</code> , <code>TransactionAmt</code> , <code>ProductCD</code> , <code>addr1</code> , <code>addr2</code> , <code>ts</code> , <code>channel</code> , <code>risk_score</code>	Raw transaction ledger from July to Dec 2016. Disguised financial exposure & timestamps.
identity.csv	144,432 records (26.7 MB)	<code>TransactionID</code> , <code>DeviceInfo</code> , <code>id_30</code> (OS), <code>id_31</code> (Browser), <code>id_33</code> (Screen), <code>DeviceType</code> , <code>id_23</code> (Proxy)	Online client hardware and network fingerprinting. Detects device-sharing & proxy usage.
closed_cases_history.csv	5,565 historical cases (2.7 MB)	<code>case_id</code> , <code>customer_id</code> , <code>card_id</code> , <code>opened_at</code> , <code>closed_at</code> , <code>outcome</code> , <code>pattern</code> , <code>exposure_usd</code> , <code>analyst_notes</code>	Institutional case memory used for graph similarity matching and pattern precedents.
case_pack.csv	20 benchmark cases (HHG-001 to HHG-020)	<code>case_id</code> , <code>opened_at</code> , <code>trigger_type</code> , <code>flagged_txn_id</code> , <code>card_id</code> , <code>customer_id</code> , <code>risk_score</code>	Official hackathon benchmark evaluation cases.

3. Live TigerGraph Cloud Infrastructure

A fresh TigerGraph Savanna v4 cluster was provisioned on AWS `us-east-1` and integrated into the project:

- **Live Host:** `https://tg-b9ee7f87-0777-4430-bc0b-f618ef476154.tg-2635877100.i.tgcloud.io`
- **Port & Routing:** Port 443 with `/restpp/` URL path prefix for REST++ endpoints and `/gsql/v1/` for GSQL statements.
- **Graph Container:** `FraudGraph`.
- **Authentication:** Production TigerGraph 4.x JWT Bearer token dynamically requested using database secret configured via environment variable (`TG_SECRET=[CONFIGURED]`) via `POST /gsql/v1/tokens`.
- **Health Status:** Latency: ~1,200 ms, Mode: `TIGERGRAPH_LIVE`, Error: `None`.

4. GSQL Graph Schema Architecture

The complete authoritative schema was defined in GSQL and compiled on the live Savanna cluster via a GSQL `SCHEMA_CHANGE JOB`.

Vertex Type	Primary ID	Attributes	Analytical Purpose
Customer	customer_id	customer_id STRING	Account holder entity
Card	card_id	customer_id, card1..card6 (network, credit/debit)	Card payment instrument
Transaction	txn_id	amount, ts, product_cd, channel, risk_score, addr1, addr2	Discrete financial movement
DeviceProfile	profile_id	device_info, os, browser, screen, device_type, is_proxy	Digital client fingerprint
EmailDomain	domain	domain STRING	Purchaser/recipient domain
BillingRegion	region_code	region_code STRING	Geographic billing jurisdiction
ClosedCase	case_id	customer_id, card_id, opened_at, outcome, pattern, exposure_usd, analyst_notes	Historical closed investigation memory
InvestigationCase	case_id	customer_id, card_id, verdict, fraud_probability, pattern, summary, sar_filed	Dynamic case written by agent loop

Edge Types (12 Directed Edge Pairs with Reverse Edges)

- `OWNS` / `OWNED_BY` (Customer → Card)
- `MADE` / `MADE_BY` (Card → Transaction)
- `FROM_DEVICE` / `DEVICE_OF` (Transaction → DeviceProfile, attribute: `device_status STRING`)
- `PURCHASER_EMAIL` / `EMAIL_OF_TXN` (Transaction → EmailDomain)
- `BILLED_IN` / `REGION_OF_TXN` (Transaction → BillingRegion)
- `NEXT` / `PREV` (Transaction → Transaction, attribute: `time_delta_sec INT`)
- `ON_CARD` / `HAS_CLOSED_CASE` (ClosedCase → Card)
- `CONNECTED_TO` / `CONNECTED_CLOSED_CASE` (ClosedCase → Card)
- `INVOLVES` / `INVOLVED_IN_CASE` (ClosedCase → Transaction)
- `INVESTIGATES` / `CASE_INVESTIGATING` (InvestigationCase → Transaction)
- `TARGET_CARD` / `INVESTIGATED_BY` (InvestigationCase → Card)
- `CONNECTED_CARD` / `CONNECTED_INVESTIGATION` (InvestigationCase → Card)

5. Pre-Compiled GSQL Queries & Graph Algorithms

Six specialized GSQL queries were authored, verified, and compiled into native C++ binaries on the live cluster:

- `card_window(VERTEX<Card> target_card, INT window_hours)` : Traverses card transactions to measure financial velocity, micro-authorization count (<\$5.00 online test charges), and total exposure.
- `device_neighbors(VERTEX<DeviceProfile> target_device)` : Traverses hardware fingerprints across multiple transactions and cards to identify multi-card syndicates and known fraudulent devices.
- `cluster_analysis(VERTEX<Customer> target_customer, STRING target_region)` : Analyzes historical geographic distributions of a customer's cards to distinguish routine travel from genuine out-of-region anomalies (Rules R2 & R3).
- `find_similar_cases(STRING target_pattern, DOUBLE min_exposure, DOUBLE max_exposure, INT max_results)` : Case-memory retrieval query filtering by typology and exposure range.
- `detect_fraud_syndicates(INT min_cards_per_cluster)` : Weakly Connected Components graph algorithm detecting clusters of cards interconnected by shared devices or emails.
- `write_case_to_graph(...)` : Dynamic agent writeback query that instantiates an `InvestigationCase` vertex, creates `TARGET_CARD` and `INVESTIGATES` edges, and records final findings in TigerGraph.

6. Live Cloud Data Ingestion

To ensure that the 20 benchmark evaluations run on genuine live TigerGraph traversals rather than fallback lookups, we ingested the entire benchmark subgraph directly into TigerGraph Savanna Cloud via high-speed REST++ batch upserts:

- **Benchmark Entities:** 20 benchmark cases, 20 customer vertices, 20 card vertices.
- **Transactions & Topology:** 1,982 transactions for benchmark cards, complete with `MADE`, `BILLED_IN`, `PURCHASER_EMAIL`, and `FROM_DEVICE` edges.
- **Hardware & Geography:** Distinct `DeviceProfile`, `BillingRegion`, and `EmailDomain` vertices.
- **Historical Memory:** 2,000 `ClosedCase` vertices with `ON_CARD` edges.
- **Total Ingested Records:** Over 10,000 vertices and edges.
- **Live Query Verification:** Running `card_window` for card `C12382-K1` on live TigerGraph traversed and returned **422 transactions and \$48,783.68 exposure** in milliseconds.

7. Python Software Architecture

The backend Python codebase is architected in a modular, decoupled hierarchy:

```
track5/
├── backend/
│   ├── config.py           # Central configuration (.env parser, paths)
│   └── tigergraph/
│       ├── client.py       # REST++ & GSQL v1 client with automatic TG 4.x JWT token management
│       ├── gateway.py      # Unified graph gateway with active engine reporting
│       ├── data_indexer.py  # High-performance SQLite B-Tree indexer (590k txns in 28s)
│       └── fallback_engine.py # Offline test harness (strictly non-authoritative)
├── scripts/
│   ├── deploy_schema.py    # Programmatic schema deployment to Savanna
│   ├── verify_schema.py    # Schema audit script verifying 8 vertices & 12 edges
│   └── load_benchmark_data_to_tigergraph.py # Cloud batch ingestion script
├── tests/
│   └── test_phase1.py      # Automated unit test suite
└── tigergraph/
    ├── schema.gsql         # Declarative GSQL schema
    ├── queries.gsql        # Declarative GSQL queries
    └── loading_jobs.gsql    # GSQL loading definitions
```

8. Verification & Test Results

All 8 unit tests in `tests/test_phase1.py` were executed against the **live TigerGraph Savanna cluster**:

```
=====
Ran 8 tests in 9.851s
OK

[ENGINE STATUS] Active Mode: TIGERGRAPH_LIVE | Host: 'https://tg-b9ee7f87-0777-4430-bc0b-f618ef476154.'
=====
```

Test Identifier	Target Feature	Execution Mode	Result
test_01_database_indexing	Local high-perf SQLite B-Tree index	Local DB	PASS
test_02_benchmark_pack_loaded	20 Benchmark Cases (HHG-001 to HHG-020)	Local DB	PASS
test_03_tigergraph_gateway_health	REST++ Echo & JWT Health Check	TigerGraph Live	PASS
test_04_card_window_query	GSQL card_window Traversal (422 txns)	TigerGraph Live	PASS
test_05_device_neighbors_query	GSQL device_neighbors Hardware Graph	TigerGraph Live	PASS
test_06_cluster_analysis_query	GSQL cluster_analysis Geographic Anomaly	TigerGraph Live	PASS
test_07_find_similar_cases_query	GSQL find_similar_cases Graph Memory	TigerGraph Live	PASS
test_08_write_case_to_graph	GSQL write_case_to_graph Dynamic Writeback	TigerGraph Live	PASS

9. Next Steps (Phases 2 through 6)

- Phase 2: TigerGraph MCP Tool Server (backend/mcp/):** Expose standardized Model Context Protocol tools for graph operations.
- Phase 3: Fraud Policy Engine (backend/policy/):** Implement deterministic rules R1 through R10 with tiered approval routing (Auto, L1, L2).
- Phase 4: Agent Investigation Loop (backend/agent/):** LangGraph 8-step investigation state machine with evidence-driven hypothesis refinement.
- Phase 5: Benchmark Evaluation:** Run cases HHG-001 to HHG-020 on live TigerGraph and produce official JSON outputs in cases/ .
- Phase 6: Frontend Workstation:** React + TypeScript console with interactive Cytoscape graph explorer and visible live health badge.